

Lab Assignment 1 – Smart Utility Toolkit

Course: Web Dev III (Node.js & Express Backend)

Unit Covered: Unit–1

Marks: 2.5

Mode: 100% In-Class (Lab Practicals)

Time: 45–60 minutes

Assignment Overview

The **Smart Utility Toolkit** is a hands-on lab assignment designed to help students understand the fundamentals of backend development using Node.js core modules.

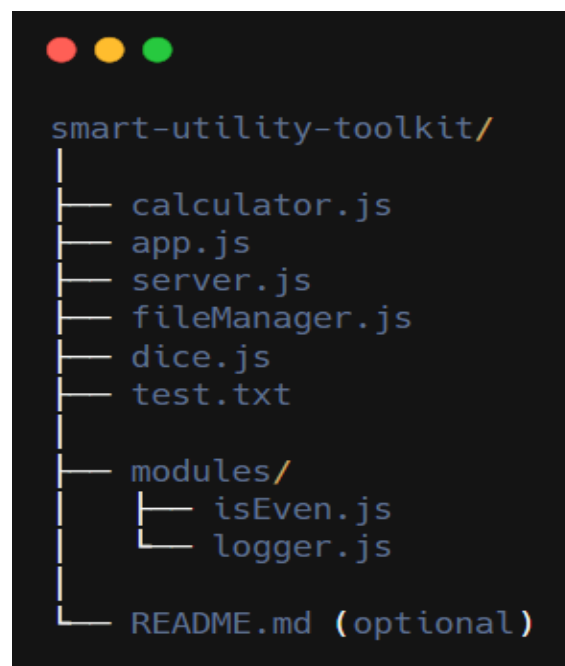
In this lab, students will build multiple mini utilities using built-in Node.js modules such as `process`, `http`, `fs`, and `crypto`, while also learning how to create and reuse custom modules in real-world scenarios.

The assignment focuses on command-line interaction, modular programming, server creation, file handling, and randomness generation—without using any external packages or frameworks.

This assignment simulates practical backend development tasks and helps students understand how Node.js works internally through hands-on experimentation.

Problem Statement

Build a simple CLI-based calculator using `process.argv` to perform operations like addition and subtraction, create a custom module (e.g., `isEven` or `logger` module) and reuse it in another file, develop a basic HTTP server using the `http` module that returns different responses on different routes, implement a file manager using `fs` module to create, read, update, and delete files, and create a random dice generator using `crypto` module to simulate real-world randomness.



Learning Objectives

- Understand how Node.js executes JavaScript outside the browser
- Use `process.argv` to take command-line inputs
- Create and reuse custom modules using `module.exports` and `require()`
- Build a basic HTTP server using the `http` core module
- Perform CRUD operations using the `fs` module
- Generate random values securely using the `crypto` module
- Understand routing concepts in backend development
- Practice modular and reusable coding practices
- Analyze server behavior and terminal outputs using console logs

Technology Stack

- **Node.js** – JavaScript runtime environment
- **JavaScript (ES6)** – Backend logic and module handling
- **Terminal / Command Prompt** – Running Node.js programs
- **Node.js Core Modules** – `process`, `http`, `fs`, `crypto`

Restrictions

- No external npm packages allowed
- No Express.js or third-party frameworks
- No database integration
- Only Node.js built-in modules should be used
- Programs must run using terminal commands

Functional Requirements

1 CLI-Based Calculator using `process.argv`

- Create a calculator that accepts command-line arguments
- Perform operations such as:
 - Addition
 - Subtraction
 - Multiplication
 - Division

Example Command: `node calculator.js add 10 5`

- Display output in terminal using `console.log()`
- Handle invalid operations gracefully

```
TERMINAL powershell + v
PS C:\Users\ASUS\Desktop\ai> node 1.js add 10 5
Result: 15
PS C:\Users\ASUS\Desktop\ai> █
```

2 Custom Module Creation & Reusability

- Create a custom module such as: isEven
- Export functionality using: module.exports
- Import the module into another file using: require()
- Demonstrate module reusability with proper examples

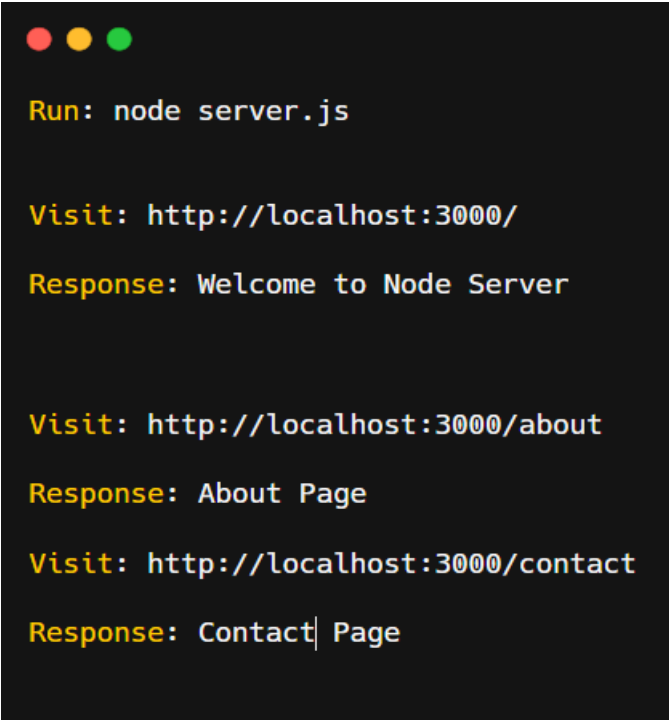
3 Basic HTTP Server using http Module

- Create a server using the built-in http module
- Run the server on a custom port

Create multiple routes such as:

Route	Response
/	Welcome message
/about	About page
/contact	Contact page
Invalid Route	404 Error Message

- Display different responses for different URLs
- Use browser or Postman to test routes



```
Run: node server.js

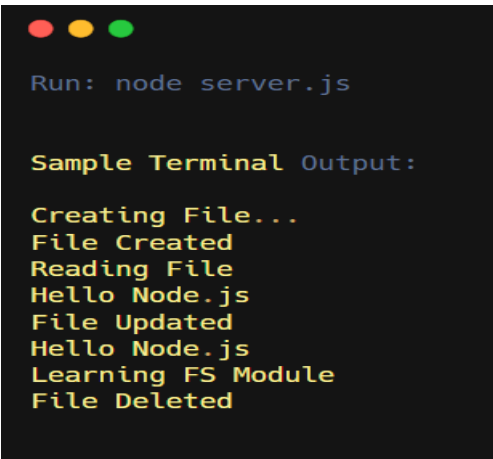
Visit: http://localhost:3000/
Response: Welcome to Node Server

Visit: http://localhost:3000/about
Response: About Page

Visit: http://localhost:3000/contact
Response: Contact Page
```

File Manager using fs Module

- Implement file operations such as:
 - Create File
 - Read File
 - Update File
 - Delete File
- Use methods such as:
 - writeFile()
 - readFile()
 - appendFile()
 - unlink()
- Display success/error messages in terminal
- Handle missing file errors gracefully



```
Run: node server.js

Sample Terminal Output:

Creating File...
File Created
Reading File
Hello Node.js
File Updated
Hello Node.js
Learning FS Module
File Deleted
```

5 Random Dice Generator using crypto Module

- Create a dice simulator that generates random numbers from 1–6
- Use the crypto module for randomness generation
- Display random dice values in terminal

Example Output:

 Dice Rolled: 4

- Simulate multiple dice rolls using loops

6 Execution Flow & Console Analysis

- Use console.log() statements to:
 - Observe execution order
 - Debug file operations
 - Analyze server requests
- Track CLI inputs
 - Display logs before and after major operations
 - Observe synchronous vs asynchronous behavior in Node.js

Bonus Challenges (Optional)

- Add colored terminal outputs using ANSI escape codes
- Add timestamp logs in the logger module
- Allow calculator to support more operations
- Store dice roll history in a text file

Rubric for Assignment–1:

Criteria	Marks
Functionality	1.5
Code Structure & Modules	0.5
Clean Code & Output	0.5
Total	2.5